DAA PRACTICAL 7

Fractional Knapsack Using Greedy Algorithm

Top 60 Viva Questions & Answers | Beginner-friendly revision guide

Core idea: Sort items by profit per unit weight in descending order. Take each item fully if it fits; otherwise take the fraction that fills the remaining capacity.

Sample result: For weights 10, 20, 30; profits 60, 100, 120; and capacity 50, take items 1 and 2 completely and 2/3 of item 3. Maximum profit = **240**.

How to use this PDF

Revise the concepts first, then practise explaining the sample dry run aloud. Pay special attention to why the ratio-based greedy choice is optimal for Fractional Knapsack but not generally optimal for 0/1 Knapsack.

A. Core concepts and algorithm choice

Q1. What is the Fractional Knapsack problem?

Answer: Given items with weights and profits and a knapsack with limited capacity, choose whole items or fractions of items to maximize total profit without exceeding the capacity.

Q2. What does “fractional” mean in this problem?

Answer: It means an item is divisible. We may take a part of an item, such as one-half or two-thirds, and receive the same fraction of its profit.

Q3. What is the main objective?

Answer: Maximize the total profit while ensuring that the total selected weight is at most the knapsack capacity.

Q4. Which algorithm is used in this program?

Answer: A greedy algorithm. At each step, it chooses the available item with the highest profit per unit of weight.

Q5. What is a greedy algorithm?

Answer: A greedy algorithm makes the best-looking choice at the current step, according to a selection rule, and proceeds without reconsidering earlier choices.

Q6. What is the greedy selection rule here?

Answer: Choose items in descending order of profit-to-weight ratio. The highest ratio gives the greatest profit for each unit of capacity used.

Q7. What is the profit-to-weight ratio?

Answer: It is profit divided by weight: ratio = profit / weight. It measures how much profit an item provides per unit of weight.

Q8. Why do we calculate the ratio?

Answer: Items can have different weights, so comparing profit alone is misleading. The ratio gives a fair comparison of profit earned for each unit of capacity.

Q9. Why do we sort in descending order?

Answer: We want the highest ratio first. The program's comparison function returns true when a.ratio is greater than b.ratio, arranging items from largest ratio to smallest.

Q10. Why is Greedy optimal for Fractional Knapsack?

Answer: Because items are divisible. If a solution uses capacity on a lower-ratio item while a higher-ratio item is still available, replacing that weight with the higher-ratio item cannot reduce profit. Repeating this exchange leads to the ratio-ordered solution.

Q11. Does the greedy ratio method always solve 0/1 Knapsack optimally?

Answer: No. In 0/1 Knapsack, items cannot be divided. Taking a high-ratio item may prevent a better combination of whole items. Dynamic programming is a standard exact method for 0/1 Knapsack.

Q12. What is the difference between Fractional and 0/1 Knapsack?

Answer: Fractional Knapsack allows taking any fraction of an item and is optimally solved by sorting ratios greedily. 0/1 Knapsack allows either the whole item or none and generally needs methods such as dynamic programming for an exact solution.

B. Structure, sorting and C++ implementation

Q13. What is the purpose of `struct Item`?

Answer: It groups the weight, profit and ratio belonging to one item into a single user-defined data type, keeping each item's related information together.

Q14. Why use `struct` instead of separate arrays?

Answer: A structure keeps an item's weight, profit and ratio together. Separate arrays can become harder to maintain because corresponding values must always use matching indices.

Q15. Could we use a `class` instead of `struct`?

Answer: Yes. Both can group data and functions. In C++, struct members are public by default, while class members are private by default. Struct is convenient here because the program directly accesses the item's fields.

Q16. What is the role of `weight` in the structure?

Answer: It stores how much knapsack capacity the item consumes if the whole item is selected.

Q17. What is the role of `profit`?

Answer: It stores the total profit earned by selecting the complete item. If only a fraction is selected, the program earns the same fraction of this profit.

Q18. What is the role of `ratio`?

Answer: It stores profit per unit weight and is used to sort items in the greedy selection order.

Q19. Why is `double ratio` used instead of `int ratio`?

Answer: The division may produce a fractional value, such as 2.5. A double stores decimal values and avoids truncating the ratio to an integer.

Q20. Why is `(double)` used in the ratio calculation?

Answer: It converts the numerator to floating-point before division, ensuring decimal division. Without it, integer division could discard the fractional part.

Q21. What happens if weight is zero?

Answer: The ratio calculation divides by zero, which is invalid. The program should validate that every item's weight is strictly positive before calculating its ratio.

Q22. What does `bool compare(Item a, Item b)` do?

Answer: It is a comparison function supplied to sort. It returns true when item a should appear before item b; here, that means a has a higher ratio than b.

Q23. Why does the comparison use `a.ratio > b.ratio`?

Answer: The greater-than comparison requests descending order, placing the most profitable item per unit weight first.

Q24. What is the purpose of `sort(items, items + n, compare)`?

Answer: It sorts the range from the first item up to, but not including, items+n, using compare to decide the order.

Q25. Why is `items + n` used?

Answer: In this array expression, items points to the first element and items+n points one position past the last element. The sort range is half-open: [items, items+n).

Q26. Which library provides `sort()`?

Answer: The `<algorithm>` header provides the standard sorting function `std::sort`.

Q27. What does `Item items[n]` do?

Answer: It declares an array intended to store n Item records. However, a variable-length array is not part of standard C++; some compilers accept it as an extension.

Q28. What is a more portable alternative to `Item items[n]`?

Answer: Use `vector<Item> items(n)` after including `<vector>`. A vector is standard C++, manages its storage, and supports a size determined at runtime.

Q29. Why do we read all items before sorting?

Answer: The program needs each item's weight and profit to calculate its ratio. Once all ratios are known, it can sort the complete item records into greedy order.

C. Greedy selection, formulas and sample dry run

Q30. What does `double totalProfit = 0.0` represent?

Answer: It initializes the accumulated profit to zero. Double is used because taking a fraction can produce a decimal profit.

Q31. Why does the loop stop when `capacity == 0`?

Answer: When no capacity remains, no additional positive-weight item can be selected, so continuing the loop would be unnecessary.

Q32. Why check `items[i].weight <= capacity`?

Answer: This determines whether the complete item fits in the remaining capacity. If it fits, the program takes the whole item.

Q33. What happens when a complete item fits?

Answer: The program subtracts its weight from capacity and adds its full profit to totalProfit.

Q34. Why do we subtract the item's weight from capacity?

Answer: Capacity represents the unused space. Subtracting the selected item's weight updates the remaining available capacity.

Q35. What happens when the complete item does not fit?

Answer: The program takes the fraction that exactly fills the remaining capacity, adds the corresponding fraction of profit, and then sets capacity to zero.

Q36. What is the formula for the fraction taken?

Answer: fraction = remaining capacity / item weight. This gives the proportion of the item that fits.

Q37. How is profit calculated for a fraction?

Answer: fractional profit = item profit × fraction. This assumes profit is proportional to the amount of the item taken.

Q38. Why is capacity set to zero after taking a fraction?

Answer: The selected fraction is calculated to fill all remaining capacity exactly, so no space remains for another item.

Q39. Work through the sample input.

Answer: Items are (weight 10, profit 60), (20, 100), and (30, 120); capacity is 50. Their ratios are 6, 5 and 4, so this is already the greedy order.

Q40. What is the sample's selection sequence?

Answer: Take item 1 fully: weight 10, profit 60; remaining capacity 40. Take item 2 fully: weight 20, profit 100; remaining 20. Take 20/30 = 2/3 of item 3: profit 120 × 2/3 = 80.

Q41. What is the maximum profit for the sample?

Answer: 60 + 100 + 80 = 240. The knapsack is filled to weight 50, so the program displays Maximum Profit: 240.

D. Complexity and alternative algorithms

Q42. What is the time complexity?

Answer: O(n log n), dominated by sorting n items. The greedy scan after sorting takes O(n), so sorting is the dominant operation.

Q43. What is the space complexity?

Answer: The array or vector stores n items, requiring O(n) space. The sorting algorithm may use additional implementation-dependent auxiliary space; the program's item storage is O(n).

Q44. Could we solve Fractional Knapsack without sorting?

Answer: A simple alternative is repeatedly searching for the highest remaining ratio, but doing that for every selection can take $O(n^2)$. Sorting once is more efficient.

Q45. When is dynamic programming preferred?

Answer: Dynamic programming is commonly used for 0/1 Knapsack, where each item must be taken whole or left behind. It considers combinations that a simple ratio-greedy method can miss.

Q46. What is the standard DP complexity for 0/1 Knapsack?

Answer: The usual table-based algorithm takes O(nW) time and O(nW) space, where n is the number of items and W is the integer capacity. This is pseudo-polynomial, not polynomial in the bit-length of W.

Q47. What is branch and bound, and when might it be used?

Answer: Branch and bound explores candidate subsets while using bounds to prune branches that cannot beat the best solution. It can be used for exact 0/1 Knapsack, especially when pruning is effective, but its worst-case time can still be exponential.

Q48. What is a limitation of the greedy method?

Answer: Its optimality depends on the problem allowing fractions and profit being proportional to the fraction taken. It is not generally optimal when items are indivisible or there are extra constraints.

E. Applications, limitations and tricky viva questions

Q49. Give real-world applications of the Fractional Knapsack model.

Answer: It can model divisible cargo, allocating limited raw materials, distributing bandwidth, or assigning divisible resources when value scales proportionally with quantity. Real applications may have additional constraints beyond this basic model.

Q50. What assumptions does this implementation make?

Answer: It assumes positive item weights, non-negative sensible input values, a non-negative capacity, and that each item's profit is proportional to the fraction selected.

Q51. How could the program be improved?

Answer: Validate n, capacity and positive weights; use vector<Item> for standard C++; use long long if integer weights or profits may exceed int; and handle invalid input and large numeric values carefully.

Q52. Why might `long long` be useful?

Answer: It provides a wider integer range than int for large weights or profits. The ratio and accumulated fractional profit should still use a suitable floating-point type when fractional results are possible.

Q53. Why is `double` used for total profit?

Answer: A fractional item can contribute a non-integer profit, so an integer accumulator would truncate or lose the fractional part. Double can represent such decimal results approximately.

Q54. Does `sort()` change the original order of items?

Answer: Yes. It rearranges the array's Item records in place according to the comparator. Each record's weight, profit and ratio move together because the complete structure is sorted.

Q55. What if two items have the same ratio?

Answer: Either order gives the same profit per unit weight for those items under the basic model. A tie-breaker may be added for deterministic ordering, but it is not required for optimality.

Q56. What if the total weight of all items is less than capacity?

Answer: The algorithm takes every item completely. The knapsack may remain partly empty because there are no more items to select.

Q57. What if capacity is zero initially?

Answer: No item can be selected. The loop immediately breaks and total profit remains zero.

Q58. What if capacity is greater than or equal to the sum of all item weights?

Answer: Every item is taken completely, and the maximum profit is the sum of all item profits, assuming positive weights and non-negative profits.

Q59. Why is the greedy choice safe even when the next item is only partly taken?

Answer: Because the remaining capacity is filled with the highest ratio still available. Any same amount of weight taken from a lower-ratio item would yield no more profit and usually less.

Q60. What should you say if asked why this practical uses Greedy rather than DP?

Answer: The items are divisible, so selecting the highest profit-to-weight ratio first is provably optimal. Sorting gives an efficient O(n log n) solution; the usual DP approach is needed for the indivisible 0/1 version, not this standard fractional version.

Formula & algorithm sheet

Purpose	Formula / rule
Profit-to-weight ratio	r■ = p■ / w■
Fraction selected	f = remaining capacity / item weight
Profit from a fraction	fractional profit = p■ × f
Total profit	Sum of full-item profits and fractional-item profit
Greedy order	Sort by ratio in descending order

Pseudocode

1. Read n, capacity and each item's weight and profit.
2. Calculate ratio = profit / weight for every item.
3. Sort items by ratio from highest to lowest.
4. Set totalProfit = 0.
5. Visit items in sorted order: take a whole item if it fits; otherwise take capacity / itemWeight of it, add that fraction of its profit, and stop.
6. Display totalProfit.

Complexity and comparison

Method	Typical role	Complexity / note
Greedy ratio sort	Fractional Knapsack	O(n log n); optimal when divisible and value is proportional
Dynamic programming	0/1 Knapsack	O(nW) time and O(nW) table space; pseudo-polynomial
Brute force subsets	Small 0/1 instances	O(2■) subsets; expensive as n grows
Branch and bound	Exact 0/1 search	Can prune branches; worst case exponential

Last-minute recall

• Fractional means items can be divided; 0/1 means whole item or nothing.

• Ratio = profit / weight. Sort highest ratio first.

• Take a full item if it fits; otherwise take exactly the fraction that fills remaining capacity.

• Greedy is optimal here because exchanging weight toward a higher ratio cannot reduce profit.

• Ratio-greedy is not guaranteed optimal for 0/1 Knapsack; DP is a standard exact approach.

• Time: O(n log n). Item storage: O(n).

• Validate positive weights to avoid division by zero; use vector<Item> for standard C++ portability.

• Sample result to remember: total profit = 240.